

Nithiyasini (192572082)

Data Structures

GPU Bitonic Sort (CUDA) VS CPU Parallel Quick Sort. (64 Cores).

Introduction:

Sorting 1 billion integers is a large-scale parallel computing task. GPU Bitonic Sort and CPU Parallel Quick Sort use different architectures to achieve high performance.

Time Complexity:

GPU Bitonic Sort:

Parallel Complexity: $O(\log^2 n)$

for $n = 10^9 \Rightarrow \log(10^9) \approx 30$.

It is suitable for massive parallel execution.

CPU Parallel Quick Sort:

Complexity $\Rightarrow O\left(\frac{n \log n}{p}\right)$

It is limited by the number of CPU Cores.

~~Memory~~ Hierarchy:

(GPU):

Global memory: Large but high latency.

shared memory: Small, on-chip, very fast.

High memory bandwidth (Hundreds of GB/s).

CPU :

- *). Uses the L1, L2, L3 cache and RAM.
- *). Lower bandwidth but better cache management.

Warp Divergence:

GPU :

- *). Threads execute in groups called warps (32 threads).
- *). Different execution paths cause warp divergence and reduce performance.
- *). Bitonic Sort minimizes divergences due to regular computation patterns.

CPU :

- *). No warp divergence.
- *). Independent cores and branch prediction reduce penalties.

PCIe Transfer Overhead :

For 1 billion Integers :

$$10^9 \times 4 = 4 \text{ GB}$$

Input + output $\Rightarrow$ 8 GB.

Data must be transferred between GPU and CPU through PCIe, adding overhead.

2. Linear Probing vs Cuckoo Hashing (100 million keys).

Introduction :

For a workload with 90% lookups and 10% inserts, hash table lookup performance is critical.

Linear Probing:

Collision handled by checking the next available slot.

Average lookup: $O(1)$

Worst case lookup: $O(n)$

Suffers from clustering.

Cuckoo Hashing:

Uses two hash functions.

Each key can be stored in one of two locations.

Worst case lookup: $O(1)$

Lookup requires checking the most two positions.

Insertion Failures:

Linear Probing:

Rare Insertion failures. Only problematic when table becomes nearly full.

Cuckoo Hashing:

Insertions may create cycles, may require rehashing with new hash functions.

Rehashing Cost:

Both methods may require $O(n)$ rehashing cost.

However, Cuckoo hashing may rehash more often because of insertion cycles.

Conclusion:

Cuckoo Hashing is preferred because it guarantees $O(1)$ worst case lookup time, providing more predictable latency.